

EE-170L Computer Programming Lab
Laboratory Session 12

Pointers in C++

Name:	Shahzad Gul
Semester:	2
Section:	B
Reg No:	0671
Lab:	12
Date:	Spring 2026

1. Objective

The objective of this lab session is to understand and apply the concepts of pointers in C++ programming. Students will learn how computer memory works, how variables are stored in memory, and how pointers can be used to access and manipulate data indirectly. By the end of this lab, students will be able to declare, initialize, and use pointers, as well as perform pointer arithmetic.

Key learning outcomes include:

- Understanding computer memory and how variables are stored
- Declaring and initializing pointer variables
- Using the referencing operator (&) to get the address of a variable
- Using the dereferencing operator (*) to access the value at a memory address
- Performing pointer arithmetic to access array elements

2. Theory

2.1 Computer Memory and Variables

Essentially, the computer's memory is made up of bytes. Each byte has a number, an address, associated with it. A variable in a program is something with a name, the value of which can vary. The compiler assigns a specific block of memory within the computer to hold the value of that variable. The size of that block depends on the data type. For example, on 32-bit PCs, the size of an integer variable is 4 bytes. When a variable is used, two distinct steps occur: first, the program finds and grabs the address reserved for that variable; second, the contents stored at that address are retrieved.

2.2 Pointers in C++

In C++, a pointer is a variable that points to or references a memory location in which data is stored. A pointer holds the memory address of another variable rather than holding a value directly. Because a pointer holds an address rather than a value, it has two parts: the pointer itself holds the address, and that address points to a value.

2.3 Pointer Declaration

The syntax for declaring a pointer is: **dataType *variableName;**. The asterisk (*) tells the compiler that you are creating a pointer variable. The type refers to the type of data stored at the address that will be stored in the pointer. For example:

```
int *ptr;    // ptr is a pointer to an integer
int k = 5;   // k is an integer variable
ptr = &k;    // ptr now holds the address of k
```

2.4 Referencing Operator (&)

The unary & operator (address-of operator) retrieves the address of a variable and copies it to the pointer. For example, **ptr = &k;** stores the address of variable **k** in pointer **ptr**. Now **ptr** is said to 'point to' **k**.

2.5 Dereferencing Operator (*)

The dereferencing operator * is used to access the value stored at the address held by the pointer. For example, ***ptr = 7;** will copy 7 to the address pointed to by **ptr**. Similarly, **cout << *ptr;** prints the value stored at that address. When we use * this way, we are referring to the value of that which **ptr** is pointing to, not the value of the pointer itself.

2.6 Pointer Arithmetic

Pointer arithmetic allows us to perform operations on pointers to navigate through memory. When we add 1 to a pointer, it does not simply add 1 to the address; instead, it adds the size of the data type the pointer points to. For example, if **ptr** is an integer pointer, **ptr + 1** adds 4 bytes (size of int) to the address. This makes pointer arithmetic ideal for traversing arrays.

Common Pointer Operations:

Operation	Description
&variable;	Returns the memory address of the variable (referencing)
*pointer	Accesses the value at the address stored in the pointer (dereferencing)
ptr++	Moves pointer to the next memory location of its data type
ptr + i	Moves pointer i positions forward in memory
*(ptr + i)	Accesses the value at the i-th position from the current pointer location

3. Lab Tasks

Task 1: Pointer Declaration and Initialization

Objective: Declare an integer variable called **num** and initialize it with a value of your choice. Declare a pointer variable called **ptr** of type integer. Initialize the pointer variable to point to the address of the **num** variable. Display the value of **num** and the value pointed to by **ptr** using the dereference operator.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 12 - Task 1: Pointer Declaration and Initialization
 * Description: Declare an integer variable, declare a pointer to it,
 *              initialize the pointer, and display values using dereferencing.
 */

#include <iostream>
using namespace std;

int main()
{
    // Declare an integer variable called num and initialize it
    int num = 25;

    // Declare a pointer variable called ptr of type integer
    int *ptr;

    // Initialize the pointer variable to point to the address of num
    ptr = &num;          // & is the referencing operator (address-of operator)

    // Display the value of num directly
    cout << "Value of num (direct access): " << num << endl;

    // Display the address of num
    cout << "Address of num: " << &num << endl;

    // Display the value stored in ptr (which is the address of num)
    cout << "Value of ptr (address stored): " << ptr << endl;

    // Display the value pointed to by ptr using dereference operator *
    cout << "Value pointed to by ptr (*ptr): " << *ptr << endl;

    return 0;
}
```

Expected Output:

Value of num (direct access): 25

```
Address of num: 0x7ffd1234abcd  
Value of ptr (address stored): 0x7ffd1234abcd  
Value pointed to by ptr (*ptr): 25
```

Explanation: The program declares an integer variable **num** initialized to 25 and a pointer **ptr** of type integer. Using the **&** operator, the address of **num** is stored in **ptr**. The program then displays the direct value of **num**, the memory address of **num** (which is the same as the value stored in **ptr**), and the dereferenced value ***ptr** which gives the value at that address. This demonstrates the relationship between a variable, its address, and pointer dereferencing.

Task 2: Pointer Arithmetic

Objective: Declare an integer array called **numbers** with some values of your choice. Declare a pointer variable called **ptr** of type integer and initialize it to point to the first element of the array. Use pointer arithmetic to access and display the elements of the array using the dereference operator.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 12 - Task 2: Pointer Arithmetic
 * Description: Demonstrates pointer arithmetic to access array elements.
 *              Pointer arithmetic automatically adjusts based on data type size.
 */

#include <iostream>
using namespace std;

int main()
{
    // Declare an integer array called numbers with some values
    int numbers[] = {10, 20, 30, 40, 50};

    // Calculate the size of the array
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // Declare a pointer variable called ptr of type integer
    int *ptr;

    // Initialize ptr to point to the first element of the array
    ptr = numbers;      // array name itself is the address of first element
                       // equivalent to: ptr = &numbers[0];

    cout << "Array elements accessed using pointer arithmetic:" << endl;
    cout << "=====" << endl;

    // Use pointer arithmetic to access and display elements of the array
    for (int i = 0; i < size; i++)
    {
        // ptr + i moves the pointer i positions forward
        // *(ptr + i) dereferences to get the value at that position
        cout << "Element " << i << ": " << *(ptr + i) << endl;
    }

    cout << "\nDemonstrating pointer increment:" << endl;
    cout << "=====" << endl;

    // Reset ptr to point to first element again
    ptr = numbers;
```

```

// Access elements using pointer increment (ptr++)
for (int i = 0; i < size; i++)
{
    cout << "Element " << i << ": " << *ptr << endl;
    ptr++;          // move pointer to next integer position
                   // automatically adds sizeof(int) bytes
}

return 0;
}

```

Expected Output:

```

Array elements accessed using pointer arithmetic:
=====
Element 0: 10
Element 1: 20
Element 2: 30
Element 3: 40
Element 4: 50

```

```

Demonstrating pointer increment:
=====
Element 0: 10
Element 1: 20
Element 2: 30
Element 3: 40
Element 4: 50

```

Explanation: The program declares an integer array **numbers** with 5 elements. A pointer **ptr** is initialized to point to the first element of the array. In the first loop, pointer arithmetic ***(ptr + i)** is used to access each element by adding an offset to the base address. In the second loop, the pointer is incremented using **ptr++**, which automatically moves the pointer to the next integer position in memory. This demonstrates that pointer arithmetic automatically adjusts based on the size of the data type (4 bytes for int).

4. Conclusion

In this lab session, we successfully explored the concept of pointers in C++. We learned how computer memory is organized into bytes with unique addresses, and how variables occupy specific memory blocks. The lab tasks demonstrated practical applications of pointer concepts, including:

- Declaring and initializing pointer variables using the `*` operator
- Using the `&` (referencing) operator to obtain the address of a variable
- Using the `*` (dereferencing) operator to access the value at a memory address
- Performing pointer arithmetic to traverse arrays efficiently

Pointers are a powerful feature of C++ that allow direct memory manipulation and efficient array handling. Understanding pointers is essential for dynamic memory allocation, data structures such as linked lists and trees, and low-level system programming. The skills learned in this lab form the foundation for advanced C++ programming concepts.

5. References

- Deitel, P. & Deitel, H. (2017). *C++ How to Program* (10th ed.). Pearson.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- UET Peshawar, EE-170L Computer Programming Lab Manual, Spring 2026.